Pontifícia Universidade Católica do Paraná – PUCPR
Escola Politécnica – Campus Sede – Curitiba/PR
Curso Bacharelado em Ciência da Computação
Disciplina de Complexidade de Algoritmos
Abílio Pedro Alcântara Mota Batista

PjBL – TRABALHO FINAL: DEDUPLICAÇÃO DE LEITURAS DE SENSORES

1 – Recomendação final por cliente

Para a **Embarca Ltda.**, recomendamos a ordenação in-place com **heapsort** seguida de varredura de vizinhos (estratégia 2 do catálogo). Ela usa **159 bytes** de memória extra, sendo 0 de heap, abaixo do limite de 256 bytes, e processa 10^7 leituras em 1,94 s.

Para o **Datacenter S.A.**, recomendamos o **bitmap** de 65 536 bits sobre o domínio conhecido (estratégia 5). Ele processa 10^7 leituras em 9,82 ms, cerca de **204 vezes abaixo** do limite de 2 s, usando 8 192 bytes de bitmap.

Cliente	Estratégia recomendada	Resultado medido (n = 10^7)	Restrição
Embarca Ltda.	2. Ordenação in-place (heapsort) + varredura	159 bytes extras (heap 0), 1,94 s	256 bytes: cumpre
Datacenter S.A.	5. Bitmap de 65 536 bits	9,82 ms, 8 192 bytes de bitmap	< 2 s: cumpre

2 – Por que essas duas estratégias

Elas ficam nos dois extremos do compromisso entre tempo e espaço, e cada uma cumpre a restrição do seu cliente e quebra a do outro. Por isso não faz sentido trocá-las:

- O bitmap usa 8 192 bytes, **32 vezes** o limite da Embarca. O heapsort leva 1,94 s para 10^7 leituras, no limite do Datacenter (nas 5 execuções fixadas em um núcleo de desempenho, 7 de 25 rodadas passaram de 2 s, até 3,31 s), e é **198 vezes** mais lento que o bitmap.
- **Alternativas descartadas para a Embarca:** só a força bruta e a ordenação cabem em 256 bytes. A força bruta leva **102 s medidos** para 10^7 leituras (321 s pela extrapolação com o teto do domínio e 13,6 h pela teoria geral $n^2/2$), cerca de 52 vezes o tempo do heapsort. Dentro da ordenação, o mergesort exige um vetor auxiliar de 20 MB e o quicksort cerca de 740 bytes de pilha; só o heapsort, iterativo e in-place, cabe. Hash, AVL, Bloom e bitmap precisam de kilobytes.
- **Alternativas descartadas para o Datacenter:** hash, AVL e Bloom também cabem em 2 s, mas o hash não tem garantia de pior caso ($O(n^2)$), o AVL encosta nos 2 s na estimativa conservadora e o Bloom perde dados. O bitmap é exato, determinístico e o mais rápido.

A análise das dez estratégias (tempo, espaço, melhor/médio/pior caso, aceitação por cliente e rejeição numérica das quatro absurdas) está em `docs/analise_teorica.md`.

3 – Resultados medidos

Cada ponto é a mediana de 5 rodadas, sobre cópias novas de dados pseudoaleatórios com semente fixa. A memória é a soma do pico de heap (alocador com contador) e do pico de pilha (técnica da pintura, um limite superior). O vetor de entrada não conta. **Máquina de medição:** notebook com Intel Core Ultra 5 125H (14 núcleos / 18 threads; L2 14 MB, L3 18 MB), 15,4 GB de RAM, Windows 11, MinGW-W64 GCC 16.1.0 com `-O2`; cada execução foi fixada em um núcleo de desempenho. Foram feitas 5 execuções completas; a tabela e o CSV trazem a de mediana. A máquina de referência da disciplina deve repetir a medição rodando o programa `main`.

n	Heapsort tempo mediano	Heapsort memória extra	Bitmap tempo mediano	Bitmap memória extra	Únicos
10^3	48,3 µs	159 B (heap 0 + pilha 159)	1,60 µs	9 271 B (heap 8 192 + pilha 1 079)	990
10^5	11,3 ms	159 B (heap 0 + pilha 159)	465 µs	9 271 B (heap 8 192 + pilha 1 079)	51 340
10^7	1,94 s	159 B (heap 0 + pilha 159)	9,82 ms	9 271 B (heap 8 192 + pilha 1 079)	65 536

Variabilidade entre as 5 execuções (mediana de cada uma): heapsort em 10^7 de 1,64 s a 2,21 s; bitmap de 7,52 ms a 9,87 ms.

Casos de teste obrigatórios:

- **Corretude:** casos com n ≤ 10, com e sem duplicatas, todas iguais, extremos 0 e 65 535, um elemento e vazio, nas duas estratégias: todos corretos. Teste adicional com 8 768 casos aleatórios e de borda contra uma referência (qsort + dedup): sem divergência.
- **Limite de memória da Embarca:** 159 bytes (heap 0 + pilha 159) em todos os n, abaixo de 256.
- **Orçamento do Datacenter:** n = 10^7 em 9,82 ms (mediana) e 10,4 ms (pior das 5 rodadas), abaixo de 2 s; pior de todas as 25 rodadas das 5 execuções: 10,4 ms.

4 – Teoria × prática e gráficos com as dez estratégias

As curvas das oito estratégias não implementadas usam o número teórico de passos multiplicado por duas constantes medidas: **K_1 = 0,98 ns** por passo simples (bitmap, n = 10^7) e **K_2 = 8,35 ns** por n·$\log_2$n (heapsort, n = 10^7). A tabela compara o tempo medido com o previsto.

n	Bitmap: medido ÷ previsto (K_1·n)	Heapsort: medido ÷ previsto (K_2·n·log n)
10^3	1,63	0,58
10^5	4,74	0,81
10^7	1,00	1,00

Os tempos seguem a ordem de grandeza da teoria (heapsort ×172 de 10^5 para 10^7, previsto ×140). As divergências foram explicadas e nenhuma muda os vereditos:

- **O bitmap fica 4,7 vezes acima da reta em n = 10^5: erro de predição de desvio, confirmado em teste.** Nesse tamanho cerca de metade das leituras é nova (51 340 únicas em 10^5), então o `if` é imprevisível. Com o mesmo conjunto de 100 000 leituras (50 000 valores, cada um duas vezes), a ordem embaralhada levou 3,6–3,8 ns por leitura contra 0,9 ns na ordem pareada e 1,9 ns em blocos: de 2 a 4 vezes mais lenta com o mesmo trabalho. Em 10^3 quase todas são novas e em 10^7 quase todas são repetidas (99,3%), e nesses casos o desvio é previsível.
- **O heapsort é 8,5 vezes mais lento que o previsto com K_1:** o passo do heapsort é mais pesado (≈ 5,4 a 5,7 ns por n·$\log_2$n mesmo com o vetor dentro do cache, contra 0,98 ns do bitmap) e, além disso, o custo sobe para 8,0 ns quando o vetor de 20 MB passa dos 18 MB do L3 (6,6 ns com 11,7 MB). Por isso as estratégias n log n usam K_2. O bitmap mexe em 8 KB, que cabem no cache.
- **A força bruta** prevê 13,6 h (n^2/2), mas como só há 65 536 valores possíveis a lista de únicas para de crescer: 321 s com K_1 e **102 s medidos em C** (0,31 ns por comparação). K_1, calibrado no bitmap, superestima em ≈ 3,1 vezes uma busca linear sequencial; o veredito não muda (51 vezes acima de 2 s mesmo medida).
- Hash, AVL e Bloom têm espaço O(n) na teoria, mas o número de distintos nunca passa de 65 536; as curvas seguem a teoria geral do catálogo.
- **Ambiente (cache, sistema operacional, núcleo, relógio):** em n = 10^3 o ruído fixo pesa (microssegundos), e em n = 10^7 o heapsort variou de 1,59 s a 3,31 s nas 25 rodadas fixadas. Em CPU híbrida, rodar em núcleo eficiente torna o tempo mais alto e variável; por isso cada execução foi fixada em um núcleo de desempenho.

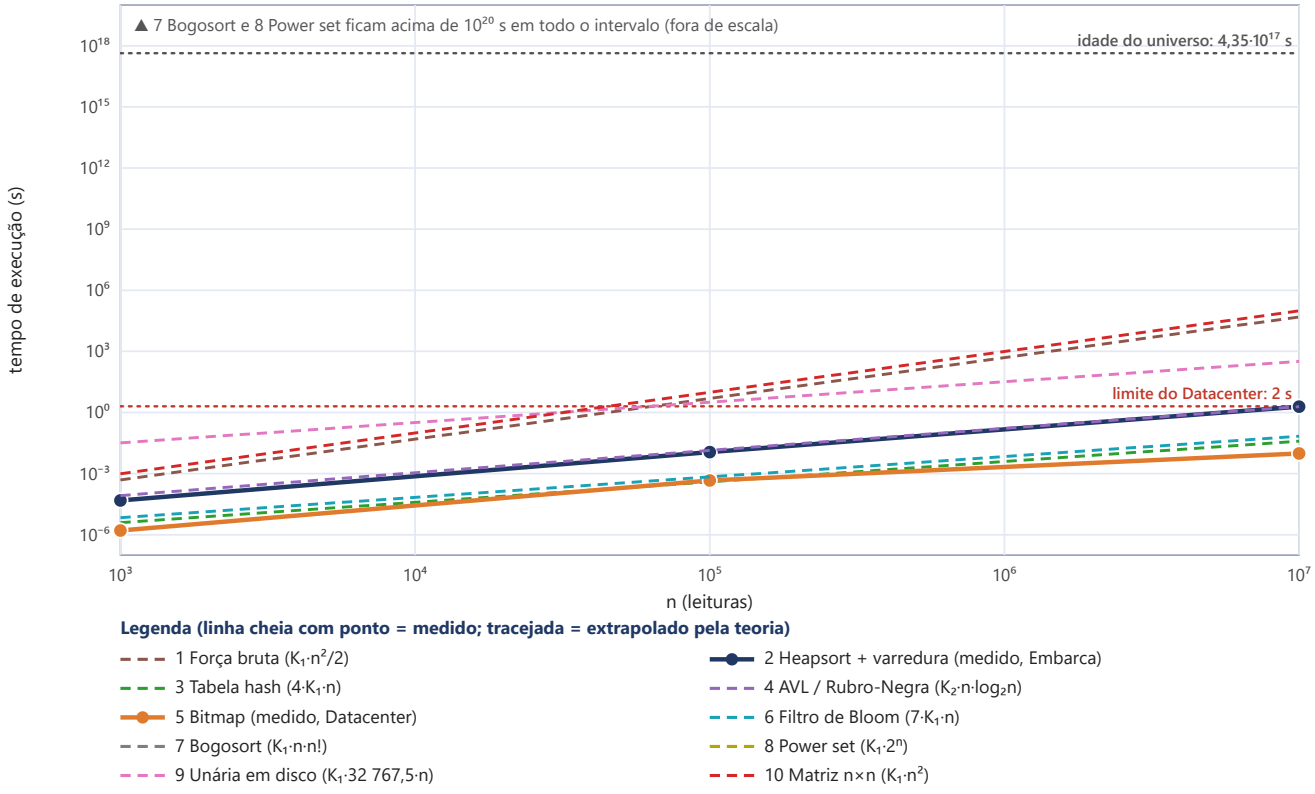


Figura 1. Tempo de execução × n (escalas logarítmicas). Linha cheia: medido. Tracejada: extrapolado pela teoria. Pontilhada: limite do Datacenter (2 s) e idade do universo.

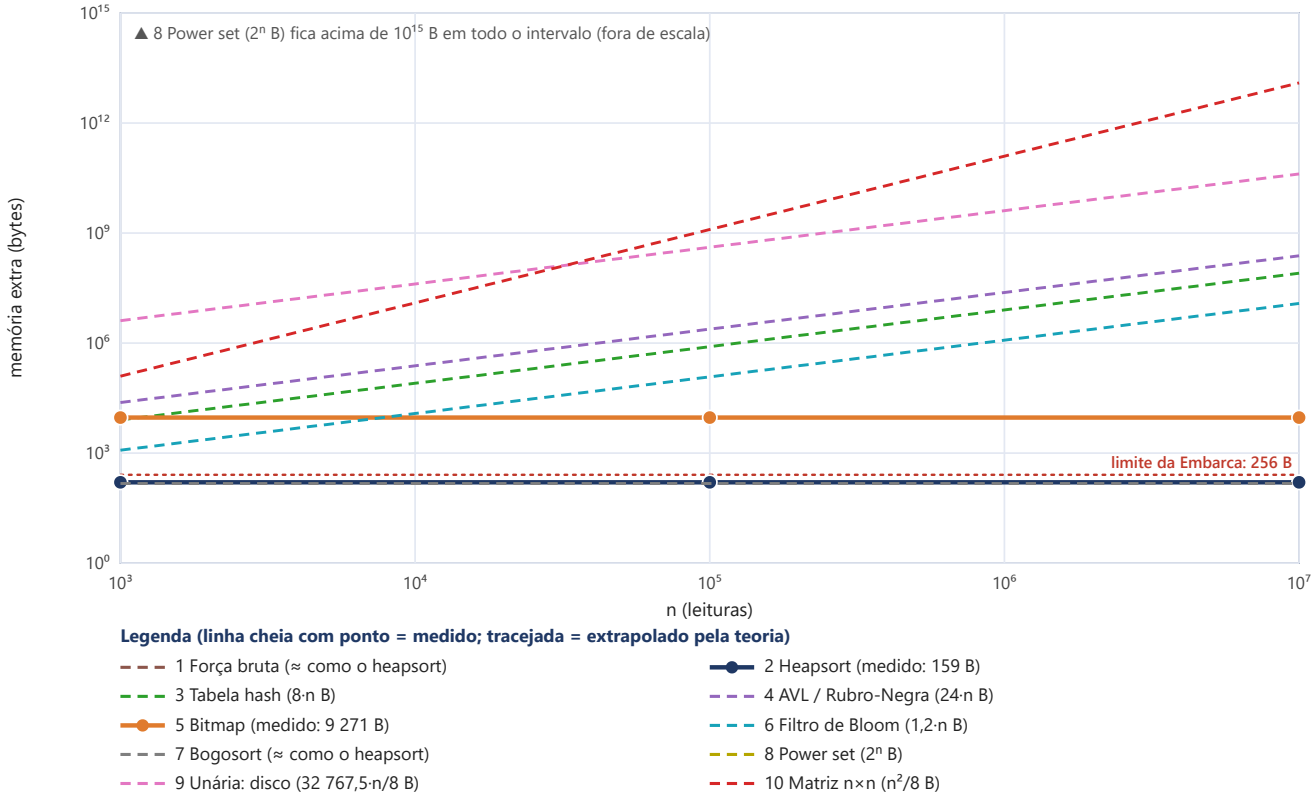


Figura 2. Espaço extra × n (escalas logarítmicas). Pontilhada: limite da Embarca (256 bytes). Força bruta e bogosort ficam em cerca de 159 bytes, como o heapsort.